The Journal of Open Source Software

ChemInformant: A Robust and Workflow-Centric Python Client for High-Throughput PubChem Access

Zhiang He[1]

1 Independent Researcher

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Evan Spotte-Smith

Reviewers:

- @cthoyt
- @Sulstice
- @LIVazquezS

Submitted: 14 April 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

ChemInformant is a modern, workflow-centric Python client designed for high-throughput and automated PubChem data tasks. Its architecture is engineered to support the direct conversion of various chemical identifiers, including mixed-type lists, into "analysis-ready" Pandas DataFrames (The pandas development team, 2025), in order to streamline the entire workflow from data retrieval to scientific analysis. As a natively integrated solution, the tool enables zero-configuration robustness by default, incorporating mechanisms such as persistent caching, automatic rate-limiting with exponential backoff, and runtime type validation with Pydantic (Colvin & others, 2025). In benchmark tests, ChemInformant's initial query was 4.6 times faster than the batch interface of a mainstream library; with a warm cache, the performance advantage increased to 48-fold, achieving response times suitable for interactive analysis tools. By systematically addressing key limitations of existing tools in network reliability, batch processing flexibility, and maintainability, ChemInformant is positioned to serve as an efficient and reliable data infrastructure component for the Python cheminformatics ecosystem.

Statement of Need

Programmatic access to PubChem (Kim et al., 2023) has become a critical infrastructure component in modern chemical and life sciences research. As research scales and automated workflows become more prevalent, researchers face three core challenges with existing client tools: the reliability of network access, the flexibility and fault tolerance of batch processing, and the sustainability risks associated with long-term library maintenance.

First, network reliability is a primary concern. PubChem's API service enforces strict dynamic rate limits (typically 5 requests per second) and returns `HTTP 503 (Server Busy)` errors under high load (National Center for Biotechnology Information, 2024). Mainstream clients like PubChemPy (Swain, 2017) generally lack built-in automatic throttling and exponential backoff retry mechanisms, which makes automated scripts brittle when faced with real-world network fluctuations. Community feedback widely reports such stability issues, with common workarounds still relying on manual insertion of delays. Furthermore, the absence of a persistent caching mechanism means that repeated queries often result in redundant network requests, increasing latency and consuming API quotas.

Second, the lack of input heterogeneity and robust error handling in batch processing presents a significant bottleneck for high-throughput research. In practical scientific tasks, users often need to process large lists containing mixed identifier types (e.g., names, CIDs, SMILES). Existing tools typically require users to manually classify and preprocess these inputs, adding complexity to the workflow. Moreover, their fault tolerance is limited: a single invalid identifier within a batch of hundreds can often cause the entire task to abort or return incomplete results without explicitly reporting which entries failed. This lack of a structured partial success/failure reporting mechanism makes error tracking difficult and compromises the transparency and reliability of the data acquisition process.

DRAFT

Finally, the maintenance status of client tools poses a potential risk to research reproducibility and stability. For instance, PubChemPy, a widely used library in the Python ecosystem, has not had a new release since version 1.0.4 in 2017, leaving community-requested features unimplemented. More importantly, a lack of active maintenance means that a tool cannot adapt to the evolution of the PubChem API itself. Users are often forced to supplement these libraries with other tools or custom logic, but such non-integrated solutions typically fail to systemically address the aforementioned stability and efficiency issues.

Therefore, a new tool that architecturally integrates robustness, efficiency, and maintainability is needed. ChemInformant is not a simple functional addition to existing tools but a holistically redesigned solution, engineered to provide the Python cheminformatics ecosystem with an extensible, high-performance, and stable data backbone that can evolve alongside upstream services.

State of the Field and Comparison

To position `ChemInformant` within the broader landscape, we examined related tools across several major platforms. **Table 1** provides a detailed comparative analysis of these tools across several dimensions critical to modern automated research.

Table 1: Comparative analysis of key features in mainstream chemical information clients.

Key Feature	**ChemInformant (v2.0)**	PubChemPy (v1.0.4)	PubChemR (v2.1.4)(Korkmaz et al., 2025)	webchem (v1.3.1)(Szöcs et al., 2020)	ChemSpiPy (v2.0.0)(Swain, 2018)	PubChem4J (Java)(Southern & Griffin, 2011)
Platform (Language)	**Python**	Python	R	R	Python	Java
Primary Database	**PubChem**	PubChem	PubChem	Multi-DB	ChemSpider	PubChem
Persistent HTTP Caching *(Improves repetitive query speed)*	**Yes (Built-in)**	No (Object-level memoization only)	No	No	No	N/A (Local DB)
Rate-Limiting & Retries *(Enhances automation robustness)*	**Yes (Built-in)**	No (Requires manual implementation)	No (Requires manual implementation)	Partial (No auto-retry)	No	N/A (Local access)
Batch Multi-Property Retrieval *(Reduces network overhead)*	**Yes (Single function, one call for multiple IDs & props)**	Partial	Partial	Partial	Partial	Yes (Local SQL)

DRAFT

Key Feature	**ChemInformant (v2.0)**	PubChemPy (v1.0.4)	PubChemR (v2.1.4)(Korkmaz et al., 2025)	webchem (v1.3.1)(Szöcs et al., 2020)	ChemSpiPy (v2.0.0)(Swain, 2018)	PubChem4J (Java)(Southern & Griffin, 2011)
Mixed Identifier Support*(Simplifies data preprocessing)*	**Yes (Native support)**	No (Requires single namespace)	No (Requires single id_type)	No (Requires single 'from' type)	No	N/A
Batch Query Fault Tolerance*(Ensures task integrity)*	**Yes (Structured status reporting)**	No (Fails on single error)	No (Silently returns NULL)	No (Silently returns NA)	No (Silently returns None)	N/A
Automatic Pagination (ListKey)*(Simplifies large dataset queries)*	**Yes (Automatic)**	Partial (Requires manual management)	No	No	No	N/A (Local SQL)
Runtime Type Safety*(Ensures data structure integrity)*	**Yes (Pydantic)**	No (No client-side validation)	Partial (R S3/S4 objects)	No	No	Yes (Static types)
Integrated 2D Structure Viz.*(Aids interactive exploration)*	**Yes (Built-in function)**	Partial (Requires manual rendering)	No	No	Partial (Requires manual rendering)	N/A
Project Activity/Status*(Ensures long-term maintainability)*	**Active**	**Inactive (since 2017)**	Active	Active	**Inactive (since 2018)**	**Archived (since 2011)**

Performance Evaluation

To quantify the performance advantages of `ChemInformant`'s architecture, we conducted a benchmark test involving the retrieval of 6 different properties for a list of 285 drug names. For a fair, same-platform comparison, we selected `PubChemPy` as the baseline. It is noteworthy that since `PubChemPy`'s batch property interface does not accept names as input, our test procedure first resolves all names to CIDs, and then times the performance of both libraries in processing

only the list of CIDs.

Table 2 summarizes the performance data for the 280 successfully resolved compounds.

Table 2: Performance benchmark results.

Scenario	Time (s)	Speed-up (vs. PubChemPy)
PubChemPy (batch interface)	6.50	1× (Baseline)
ChemInformant — Cold Cache	**1.40**	**4.6×**
ChemInformant — Warm Cache	**0.135**	**48×**

`ChemInformant`'s initial batch query completed in **1.4 seconds**, making it **4.6 times faster** than `PubChemPy`'s 6.5 seconds. A subsequent cached query finished in just **135 milliseconds**—a further 10-fold improvement, resulting in an overall 48-fold speed-up compared to the baseline. This sub-200ms response time achieves a level of latency suitable for interactive applications. The full benchmark script is available in the project repository.

Example Usage

`ChemInformant` provides a layered API, enabling both quick lookups via convenience functions and powerful batch processing through its core engine.

Convenience API Example (for quick lookups):

```python
import ChemInformant as ci

# Quickly retrieve a single property
cas_number = ci.get_cas("ibuprofen")
# > '15687-27-1'
```

Core API Example (for batch data analysis):

```python
# Retrieve multiple properties for a list of mixed-identifier types
df = ci.get_properties(
    identifiers=["aspirin", "caffeine", 1983], # Mix of names and a CID
    properties=["molecular_weight", "xlogp", "cas"]
)
# The returned DataFrame is immediately ready for downstream analysis tools
print(df)
```

A more comprehensive, interactive user manual that demonstrates how to pipe data to other analysis tools is available in the project repository.

Acknowledgements

The author thanks PubChem for providing open data services, and acknowledges the developers of the open-source libraries upon which `ChemInformant` is built, including `requests`, `requests-cache`, `pandas`, and `pydantic`. The author is also grateful for community feedback provided during the development process.

Colvin, S., & others. (2025). *Pydantic* (Version 2.11.7). Zenodo. https://doi.org/10.5281/zenodo.15662245

Kim, S., Chen, J., Cheng, T., Gindulyte, A., He, J., He, S., Li, Q., Shoemaker, B. A., Thiessen, P. A., Yu, B., Zaslavsky, L., Zhang, J., & Bolton, E. E. (2023). PubChem 2023 update. *Nucleic Acids Research*, *51*(D1), D1373–D1380. https://doi.org/10.1093/nar/gkac956

DRAFT

Korkmaz, S., Yamasan, B. E., & Goksuluk, D. (2025). *PubChemR: An r client for the PubChem API* (Version 2.1.4). Comprehensive R Archive Network (CRAN). https://doi.org/10.32614/CRAN.package.PubChemR

National Center for Biotechnology Information. (2024). *PubChem programmatic access usage policy*. https://pubchem.ncbi.nlm.nih.gov/docs/programmatic-access

Southern, M. R., & Griffin, P. R. (2011). A java API for working with PubChem datasets. *Bioinformatics*, *27*(5), 741–742. https://doi.org/10.1093/bioinformatics/btq715

Swain, M. (2017). *PubChemPy v1.0.4* (Version 1.0.4). Zenodo. https://doi.org/10.5281/zenodo.541438

Swain, M. (2018). *ChemSpiPy: A python wrapper for the ChemSpider API* (Version 2.0.0). https://pypi.org/project/ChemSpiPy/2.0.0

Szöcs, E., Stirling, T., Scott, E. R., Scharmüller, A., & Schäfer, R. B. (2020). Webchem: An r package to retrieve chemical information from the web. *Journal of Statistical Software*, *93*(13), 1–17. https://doi.org/10.18637/jss.v093.i13

The pandas development team. (2025). *Pandas-dev/pandas: pandas* (Version 2.3.1). Zenodo. https://doi.org/10.5281/zenodo.15831829

DRAFT